

Your grade: 100%

Your latest: 100% • Your highest: 100% • To pass you need at least 80%. We keep your highest score.

Next item →

1. The A* (A-star) algorithm is a popular pathfinding and graph traversal method. What is the primary achievement or goal when utilizing the A* algorithm in applications such as video games, robotics, or mapping software?

1 / 1 point

- ☐ Generating complex 3D graphics
- ☐ Enhancing the security of digital communications
- ☒ Finding the shortest path between two points
- ☐ Optimizing database queries
- ☐ Simulating realistic physical interactions in virtual environments
- ☒ Correct
- Correct: The primary goal of the A* algorithm is to find the most efficient, or shortest, path between two points in a graph or grid. This capability makes it highly valuable in applications requiring optimal routing solutions, such as navigating characters in video games, directing robots in physical spaces, or computing routes in mapping software.

2. Pathfinding algorithms, such as A* (A-star), are designed to compute the most efficient path between two points in a given environment. For these algorithms to execute their computations and deliver accurate results, what kind of setup or world configuration is typically required?

1 / 1 point

- ☐ A fully random environment with no defined start or end points
- ☐ A detailed physical model of the world including weather conditions and day-night cycles
- ☒ A graph or grid representation of the environment with defined obstacles and traversable areas
- ☐ An environment with dynamic obstacles that change position every few seconds
- ☐ A high-resolution satellite imagery overlay providing real-time data updates
- ☒ Correct
- Correct: Pathfinding algorithms typically operate on a simplified representation of the environment, such as a graph or a grid, where nodes or cells correspond to possible positions in the environment. This representation includes defined obstacles (untraversable nodes or cells) and open, traversable areas, allowing the algorithm to compute paths by navigating through connected nodes or grid cells.

3. The Flood Fill and A* (A-star) algorithms are both utilized in various computational fields for different purposes, including graphics and pathfinding, respectively. Both algorithms use a form of a "propagate" function to extend their operation across the given space. However, the functionality and purpose of the propagate function significantly differ between Flood Fill and A*. What is the primary difference between the propagate function in a Flood Fill algorithm and in an A* algorithm?

1 / 1 point

- ☒ In Flood Fill, the propagate function spreads uniformly in all directions from a starting point until it encounters boundaries, whereas in A*, the propagate function prioritizes directions based on a heuristic that estimates the cost to the target
- ☐ Flood Fill's propagate function changes the color or property of nodes, while A* function recalculates the entire grid layout for every step
- ☐ Both Flood Fill and A* use the propagate function to calculate the shortest path between two points
- ☐ The propagate function in Flood Fill relies on a heuristic to estimate costs, whereas in A*, it spreads indiscriminately without such estimates
- ☐ In Flood Fill, the propagate function is used to detect and avoid obstacles, whereas in A*, it is solely for visual effects and has no impact on pathfinding
- ☒ Correct
- Correct: The key difference lies in the decision-making process about where to propagate next. Flood Fill algorithm's propagation is uninformed and spreads equally in all accessible directions without prioritizing, while A* propagation is informed and directional, prioritizing which routes to explore next based on a heuristic estimate of the remaining cost to the target, aiming to find the most efficient path.

4. In a simulation developed with Processing using Python mode, you aim to allow users to place obstacles interactively on a grid that spans the entire canvas. This feature should enable obstacles to be added to the grid when the user clicks and drag the mouse across the canvas. Given that the position of the mouse needs to be translated to the corresponding grid cell, how can you implement the functionality to "paint" obstacles onto the grid based on mouse movement?

1 / 1 point

- ☐
- ```
1 def mousePressed():
2 col = int(mouseX / cellSize)
3 row = int(mouseY / cellSize)
4 grid[row][col] = OBSTACLE
```
- ☒
- ```
1 def mouseDragged():
2     col = int(mouseX / cellSize)
3     row = int(mouseY / cellSize)
4     if 0 < col < cols and 0 < row < rows:
5         grid[row][col] = OBSTACLE
```
- ☐
- ```
1 def mouseDragged():
2 background(180)
```
- ☐
- ```
1 def mouseDragged():
2     col = 20
3     row = 1
4     grid[row][col] = OBSTACLE
```
- ☐
- ```
1 def mouseDragged():
2 col = mouseX
3 row = mouseY
4 if 0 < col < cols and 0 < row < rows:
5 grid[row][col] = OBSTACLE
6
```

☒ Correct

Correct: This snippet is effective for painting obstacles on the grid through mouse dragging. It calculates the grid cell that corresponds to the current mouse position and updates that cell to represent an obstacle, ensuring actions are within the grid's bounds. This method provides the intended functionality of interactively modifying the environment.

5. In a grid-based environment, the A\* (A-star) algorithm is employed to find the optimal path from a designated starting cell to a target cell. What sequence of steps does A\* take to successfully determine and complete this optimal path?

1 / 1 point

- ☒ A\* starts by adding the starting cell to a priority queue based on the cost function  $f = g + h$ , where  $g$  is the cost from the start to the current cell, and  $h$  is the estimated cost from the current cell to the target. It then iterates, choosing the cell with the lowest  $f$  cost, updates neighbors' costs, and tracks the best path until it reaches the target.
- ☐ A\* randomly selects neighboring cells to explore from the starting cell, gradually increasing the search radius until the target cell is found.
- ☐ A\* first calculates the total distance from the start cell to every other cell in the grid, then selects the shortest path from these distances to reach the target.
- ☐ A\* marks all cells as visited starting from the target cell and moves outward in all directions until it finds the starting cell.
- ☐ A\* uses a depth-first search strategy from the starting cell, backtracking whenever a dead end is reached, until the target cell is found.
- ☒ Correct
- Correct: This choice accurately describes the process of the A\* algorithm. A\* initializes by considering the starting cell and then iteratively explores the most promising cells (those with the lowest  $f$  costs). It updates the costs ( $g$ ,  $h$ , and consequently  $f$ ) of neighboring cells and keeps track of the most efficient path taken to reach each cell by using a priority queue to efficiently select the next cell to explore. The process continues until the target cell is reached, ensuring the path found is optimal.

6. When visualizing the execution of the A\* (A-star) algorithm in a grid-based pathfinding application, it's crucial to have clear indicators that the algorithm's calculations are proceeding as expected. What kind of graphical data could be displayed within each visited cell to help check that the calculation of the A\* algorithm is being done correctly?

1 / 1 point

- ☐ The RGB color values of each cell based on its distance from the start
- ☐ The direction of the optimal path as an arrow in each cell
- ☒ The  $g$ ,  $h$ , and  $f$  values displayed as numbers in each visited cell
- ☐ The total number of cells visited up to that point displayed in each cell
- ☐ A binary indicator showing whether a cell is an obstacle or free space.
- ☒ Correct
- Correct: Displaying the  $g$  and  $h$  values as numbers in each visited cell provides a direct way to verify the A\* algorithm's calculations. These values are central to the algorithm's decision-making process, and visualizing them allows for an immediate check of whether the algorithm is evaluating and updating cell costs correctly as it searches for the optimal path.

7. In the A\* (A-star) algorithm, when a new cell is visited during the search for the optimal path to the target, certain information must be stored to facilitate the reconstruction of the path once the target has been reached. What specific kind of information does the A\* algorithm need to keep for each visited cell to enable accurate path reconstruction from the target back to the starting point?

1 / 1 point

- ☐ The exact distance from the start cell to the visited cell
- ☐ The color or visual representation of the cell
- ☒ Each visited cell must store a reference to its predecessor cell (or parent cell), the cell from which it was directly reached during the search.
- ☐ The total number of cells visited by the algorithm up to that point
- ☐ The estimated time to reach the target from the current cell
- ☒ Correct
- Correct: Storing a reference to the parent cell (the cell from which the current cell was discovered) for each visited cell is crucial for path reconstruction. Once the target is reached, the algorithm can trace back through these parent links to reconstruct the path from the target back to the start, effectively reversing the discovered route.

8. Consider the process of implementing the A\* search algorithm in a grid-based environment using Processing with Python mode. Below are simplified versions of the A\* search function. Each aims to refine the algorithm's core logic, from starting the search to finding the path to the target node. Select the version that correctly implements the fundamental steps of the A\* algorithm.

1 / 1 point

- ☐
- ```
1 def a_star_search():
2     while not at_target():
3         current = get_node_with_min_f_cost_distance()
4         if current is target:
5             print("Target reached.")
6             return
7         mark_visited(current)
```
- ☐
- ```
1 def a_star_search():
2 while not at_target():
3 current = select_random_neighbor()
4 if current is target:
5 print("Target reached.")
6 return
7 mark_visited(current)
```
- ☐
- ```
1 def a_star_search():
2     for node in all_nodes:
3         if node is target:
4             print("Target reached.")
5             return
6         mark_visited(node)
```
- ☒
- ```
1 def a_star_search():
2 open_set = initialize_with_start_node()
3 while open_set:
4 current = get_node_with_lowest_f_cost(open_set)
5 if current is target:
6 print("Goal Reached!")
7 reconstruct_path(current)
8 return
9 open_set.remove(current)
10 for neighbor in get_neighbors(current):
11 if is_valid(neighbor):
12 update_g_cost_and_parent(neighbor, current)
13 if neighbor not in open_set:
14 open_set.append(neighbor)
```
- ☐
- ```
1 def a_star_search():
2     open_set = initialize_with_start_node()
3     while open_set:
4         current = get_node_with_lowest_f_cost(open_set)
5         if current is target:
6             print("Goal Reached!")
7             reconstruct_path(current)
8             return
9         open_set.remove(current)
10        for neighbor in get_neighbors(current):
11            if is_valid(neighbor):
12                update_g_cost_and_parent(neighbor, current)
13                if neighbor not in open_set:
14                    open_set.append(neighbor)
15
```

☒ Correct

Correct: This version correctly encapsulates the essence of the A* search algorithm. It prioritizes nodes based on the estimated total cost ($f = g + h$), explores neighbors, updates their costs, and reconstructs the path upon reaching the target, reflecting A*'s strategic exploration based on cost efficiency.